NAME: AYAS AHAMED

ROLL NO: 2025115162

1. Simulate **Breadth-First Search (BFS)** and **Depth-First Search (DFS)** traversal for the given graph.

Code:

```
#include <stdio.h>

#define MAX 100

int graph[MAX][MAX];
int visited[MAX];
int n;

// BFS
void BFS(int start) {
    int queue[MAX];
    int front = 0, rear = 0;

    for (int i = 0; i < n; i++)
        visited[i] = 0;

    queue[rear++] = start;
    visited[start] = 1;

    printf("BFS: ");

    while (front < rear) {
        int v = queue[front++];
        printf("%d ", v);

        for (int i = 0; i < n; i++) {
            if (graph[v][i] && !visited[i]) {
```

```
                visited[i] = 1;
                queue[rear++] = i;
            }
        }
    }
}

// DFS
void DFS(int v) {
    visited[v] = 1;
    printf("%d ", v);

    for (int i = 0; i < n; i++) {
        if (graph[v][i] && !visited[i])
            DFS(i);
    }
}

int main() {
    int e, u, v, start;

    printf("Enter vertices and edges: ");
    scanf("%d %d", &n, &e);

    if (n < 1 || n > MAX || e < 0)
        return 1;

    for (int i = 0; i < e; i++) {
        scanf("%d %d", &u, &v);

        if (u < 0 || u >= n || v < 0 || v >= n)
            return 1;
```

```
        graph[u][v] = 1;
        graph[v][u] = 1;
    }

    printf("Enter starting vertex: ");
    scanf("%d", &start);

    if (start < 0 || start >= n)
        return 1;

    BFS(start);

    for (int i = 0; i < n; i++)
        visited[i] = 0;

    printf("\nDFS: ");
    DFS(start);

    return 0;
}
```

Sample Input

```
7 8
0 1
0 2
1 3
1 4
2 5
2 6
3 4
5 6
0
```

Output

BFS: 0 1 2 3 4 5 6

DFS: 0 1 3 4 2 5 6

2. Find the Minimum Spanning Tree using Prim's and Kruskal's algorithms.

Code:

```
#include <stdio.h>
#include <limits.h>

#define MAX 100

int main() {
    int n, graph[MAX][MAX];
    int selected[MAX] = {0};
    int cost = 0;

    printf("Enter number of vertices: ");
    scanf("%d", &n);

    if (n < 1 || n > MAX)
        return 1;

    printf("Enter adjacency matrix:\n");

    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            scanf("%d", &graph[i][j]);

    selected[0] = 1;

    printf("MST Edges:\n");

    for (int count = 0; count < n - 1; count++) {
```

```
    int min = INT_MAX;
    int x = -1, y = -1;

    for (int i = 0; i < n; i++) {
        if (selected[i]) {
            for (int j = 0; j < n; j++) {
                if (!selected[j] &&
                    graph[i][j] > 0 &&
                    graph[i][j] < min) {

                    min = graph[i][j];
                    x = i;
                    y = j;
                }
            }
        }
    }

    if (y == -1) {
        printf("Graph is disconnected\n");
        return 0;
    }

    printf("%d - %d : %d\n", x, y, min);

    selected[y] = 1;
    cost += min;
}

printf("Minimum Cost = %d\n", cost);

return 0;
```

```
}
```

3. The edge labels in the photographs are not all clear enough to guarantee an accurate numerical MST. Rather than inventing weights, use the following program to calculate the MST for each graph.

Code:

```
#include <stdio.h>

#include <stdlib.h>


#define MAX 100


struct Edge {

    int u, v, w;

};


struct Edge edges[MAX];

int parent[MAX];


int find(int x) {

    if (parent[x] != x)

        parent[x] = find(parent[x]);


    return parent[x];

}


void unite(int a, int b) {

    parent[find(a)] = find(b);

}


int compare(const void *a, const void *b) {

    int x = ((struct Edge *)a)->w;

    int y = ((struct Edge *)b)->w;


    return (x > y) - (x < y);
```

```
}

int main() {
    int n, e, cost = 0, count = 0;

    printf("Enter vertices and edges: ");
    scanf("%d %d", &n, &e);

    if (n < 1 || n > MAX || e < 0 || e > MAX)
        return 1;

    printf("Enter edges (u v weight):\n");

    for (int i = 0; i < e; i++) {
        scanf("%d %d %d",
              &edges[i].u,
              &edges[i].v,
              &edges[i].w);

        if (edges[i].u < 0 || edges[i].u >= n ||
            edges[i].v < 0 || edges[i].v >= n)
            return 1;
    }

    for (int i = 0; i < n; i++)
        parent[i] = i;

    qsort(edges, e, sizeof(struct Edge), compare);

    printf("Selected MST Edges:\n");

    for (int i = 0; i < e; i++) {
```

```
        int u = edges[i].u;
        int v = edges[i].v;

        if (find(u) != find(v)) {
            unite(u, v);

            printf("%d - %d : %d\n",
                  u, v, edges[i].w);

            cost += edges[i].w;
            count++;

            if (count == n - 1)
                break;
        }
    }

    if (count == n - 1)
        printf("Minimum Cost = %d\n", cost);
    else
        printf("MST not possible: disconnected graph\n");

    return 0;
}
```

4. Find the shortest path from a source vertex to all other vertices.

Code:

```
#include <stdio.h>
#include <limits.h>

#define MAX 100
#define INF INT_MAX
```

```
int graph[MAX][MAX];
int dist[MAX], visited[MAX];
int n;

void dijkstra(int source) {
    for (int i = 0; i < n; i++) {
        dist[i] = INF;
        visited[i] = 0;
    }

    dist[source] = 0;

    for (int count = 0; count < n; count++) {
        int u = -1;

        for (int i = 0; i < n; i++) {
            if (!visited[i] &&
                (u == -1 || dist[i] < dist[u]))
                u = i;
        }

        if (u == -1 || dist[u] == INF)
            break;

        visited[u] = 1;

        for (int v = 0; v < n; v++) {
            if (graph[u][v] > 0 &&
                !visited[v] &&
                dist[u] <= INF - graph[u][v] &&
                dist[u] + graph[u][v] < dist[v]) {
```

```
                dist[v] = dist[u] + graph[u][v];
            }
        }
    }

    printf("Vertex\tDistance\n");

    for (int i = 0; i < n; i++) {
        if (dist[i] == INF)
            printf("%d\tINF\n", i);
        else
            printf("%d\t%d\n", i, dist[i]);
    }
}

int main() {
    int source;

    printf("Enter number of vertices: ");
    scanf("%d", &n);

    if (n < 1 || n > MAX)
        return 1;

    printf("Enter adjacency matrix:\n");

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &graph[i][j]);

            if (graph[i][j] < 0) {
                printf("Negative weights not allowed\n");
```

```
            return 1;
        }
    }
}

printf("Enter source: ");
scanf("%d", &source);

if (source < 0 || source >= n)
    return 1;

dijkstra(source);

return 0;
}
```

Sample Input

```
5
0 10 3 0 0
0 0 1 2 0
0 4 0 8 2
0 0 0 0 7
0 0 0 9 0
0
```

Output

```
Vertex  Distance
0       0
1       7
2       3
3       9
4       5
```

5. Write a C program to perform topological sorting of a Directed Acyclic Graph.

Code:

```c
#include <stdio.h>

#define MAX 100

int graph[MAX][MAX];
int indegree[MAX];
int queue[MAX];

int main() {
    int n, e, u, v;
    int front = 0, rear = 0;
    int count = 0;

    printf("Enter vertices and edges: ");
    scanf("%d %d", &n, &e);

    if (n < 1 || n > MAX || e < 0)
        return 1;

    printf("Enter directed edges:\n");

    for (int i = 0; i < e; i++) {
        scanf("%d %d", &u, &v);

        if (u < 0 || u >= n || v < 0 || v >= n)
            return 1;

        if (!graph[u][v]) {
            graph[u][v] = 1;
            indegree[v]++;
        }
    }
```

```
    for (int i = 0; i < n; i++) {
        if (indegree[i] == 0)
            queue[rear++] = i;
    }

    printf("Topological Order: ");

    while (front < rear) {
        int u = queue[front++];

        printf("%d ", u);
        count++;

        for (int v = 0; v < n; v++) {
            if (graph[u][v]) {
                indegree[v]--;

                if (indegree[v] == 0)
                    queue[rear++] = v;
            }
        }
    }

    if (count != n)
        printf("\nCycle detected: topological sort not possible");

    return 0;
}
```

Sample Input

6 6

5 2

5 0

4 0

4 1

2 3

3 1

Output

Topological Order: 4 5 0 2 3 1

6. Insert the following keys into a hash table of size 7.

Code:

```
#include <stdio.h>

#define SIZE 7

int table[SIZE];
int occupied[SIZE] = {0};

void insert(int key) {
    int index = key % SIZE;

    if (index < 0)
        index += SIZE;

    for (int i = 0; i < SIZE; i++) {
        int pos = (index + i) % SIZE;

        if (!occupied[pos]) {
            table[pos] = key;
            occupied[pos] = 1;
            return;
        }
    }
```

```
    printf("Hash table full\n");
}

void display() {
    for (int i = 0; i < SIZE; i++) {
        if (occupied[i])
            printf("%d : %d\n", i, table[i]);
        else
            printf("%d : Empty\n", i);
    }
}

int main() {
    int keys[] = {76, 93, 40, 47, 10, 55};

    for (int i = 0; i < 6; i++)
        insert(keys[i]);

    display();

    return 0;
}
```

Output

```
0 : 47
1 : 55
2 : 93
3 : 10
4 : Empty
5 : 40
6 : 76
```

7. Hash table size:

$$m = 7$$

Hash function:

$$h(k) = k\,7$$

Collision resolution:

$$h(k, i) = (h(k) + i^2)\,7$$

Keys:

Code:

```
#include <stdio.h>

#define SIZE 7

int table[SIZE];
int occupied[SIZE] = {0};

void insert(int key) {
    int h = key % SIZE;

    if (h < 0)
        h += SIZE;

    for (int i = 0; i < SIZE; i++) {
        int index = (h + i * i) % SIZE;

        if (!occupied[index]) {
            table[index] = key;
            occupied[index] = 1;
            return;
        }
    }
```

```
    printf("Insertion failed for %d\n", key);
}

int main() {
    int keys[] = {22, 30, 50};

    for (int i = 0; i < 3; i++)
        insert(keys[i]);

    printf("Hash Table:\n");

    for (int i = 0; i < SIZE; i++) {
        if (occupied[i])
            printf("%d : %d\n", i, table[i]);
        else
            printf("%d : Empty\n", i);
    }

    return 0;
}
```

Output

Hash Table:

0 : Empty

1 : 22

2 : 30

3 : Empty

4 : Empty

5 : 50

6 : Empty

8. Keys:

Table size:

$$m = 7$$

First hash function:

$$h_1(k) = k\,7$$

Second hash function:

$$h_2(k) = 1 + (k\,5)$$

Double hashing formula:

Code:

Hash Table:

0 : Empty

1 : 43

2 : 72

3 : Empty

4 : 69

5 : Empty

6 : 27

9. Prove that if quadratic probing is used and the hash table size is prime, a new element can always be inserted if the table is at least half empty.

Let the hash table size be a prime number p.

Quadratic probing uses:

$$h(k, i) = (h(k) + i^2)\,p$$

Consider the probe positions for:

$$i = 0,1,2, \dots, \frac{p-1}{2}$$

We will prove that these positions are distinct.

Assume two probe positions are equal:

$$h(k) + i^2 \equiv h(k) + j^2 \pmod{p}$$

Subtract $h(k)$from both sides:

$$i^2 \equiv j^2 \pmod{p}$$

Therefore:

$$i^2 - j^2 \equiv 0 \pmod{p}$$

Factorize:

$$(i - j)(i + j) \equiv 0 \pmod{p}$$

Since $p$is prime:

$$i - j \equiv 0 \pmod{p}$$

or

$$i + j \equiv 0 \pmod{p}$$

But:

$$0 \leq i, j \leq \frac{p-1}{2}$$

Hence:

$$0 \leq i + j \leq p - 1$$

The second condition is possible only when $i = j = 0$.

The first condition also implies:

$$i = j$$

Therefore, two different values of $i$cannot produce the same probe position within this range.

Thus, quadratic probing examines:

$$\frac{p+1}{2}$$

distinct positions.

If the table is at least half empty, the number of occupied positions is at most:

$$\frac{p-1}{2}$$

Since quadratic probing checks:

$$\frac{p+1}{2}$$

distinct positions, at least one of these positions must be empty.

Therefore, insertion is guaranteed.

Hence proved.

Important: This proof assumes an odd prime table size and the standard probing function $h(k, i) = (h(k) + i^2)\, p$.

10. Implement a hash table using separate chaining with the following operations:

    1. Creation
    2. Initialization
    3. Insertion
    4. Searching

Code:

```
#include <stdio.h>
#include <stdlib.h>

#define SIZE 7

struct Node {
    int data;
    struct Node *next;
};

struct Node *table[SIZE];

// Initialize hash table
void initialize() {
    for (int i = 0; i < SIZE; i++)
        table[i] = NULL;
}

// Hash function
int hash(int key) {
```

```
    return ((key % SIZE) + SIZE) % SIZE;
}

// Create node
struct Node* createNode(int key) {
    struct Node *newNode = malloc(sizeof(struct Node));

    if (newNode == NULL) {
        printf("Memory allocation failed\n");
        exit(1);
    }

    newNode->data = key;
    newNode->next = NULL;

    return newNode;
}

// Insert
void insert(int key) {
    int index = hash(key);

    struct Node *newNode = createNode(key);

    newNode->next = table[index];
    table[index] = newNode;
}

// Search
void search(int key) {
    int index = hash(key);
```

```
    struct Node *temp = table[index];

    while (temp != NULL) {
        if (temp->data == key) {
            printf("%d found at index %d\n", key, index);
            return;
        }

        temp = temp->next;
    }

    printf("%d not found\n", key);
}

// Display
void display() {
    for (int i = 0; i < SIZE; i++) {
        printf("%d: ", i);

        struct Node *temp = table[i];

        while (temp != NULL) {
            printf("%d -> ", temp->data);
            temp = temp->next;
        }

        printf("NULL\n");
    }
}

int main() {
    int choice, key;
```

```
initialize();

while (1) {
    printf("\n1. Insert");
    printf("\n2. Search");
    printf("\n3. Display");
    printf("\n4. Exit");

    printf("\nEnter choice: ");
    scanf("%d", &choice);

    switch (choice) {
        case 1:
            printf("Enter key: ");
            scanf("%d", &key);
            insert(key);
            break;

        case 2:
            printf("Enter key to search: ");
            scanf("%d", &key);
            search(key);
            break;

        case 3:
            display();
            break;

        case 4:
            return 0;
```

```
            default:
                printf("Invalid choice\n");
        }
    }
}
```

Sample Input

Insert:

10 17 24 5 12

Final Hash Table

Index 0: NULL

Index 1: NULL

Index 2: NULL

Index 3: 24 -> 17 -> 10 -> NULL

Index 4: NULL

Index 5: 12 -> 5 -> NULL

Index 6: NULL